Практична робота №7

Експерименти з підвищеним навантаженням: оптимізація серверних скриптів

ІО-43 Череватов Артем

Варіант 20

Крок 1. Вимірювання базового часу виконання «як є»

Для вимірювання базових показників у файл api.php для ендпоінта resource=teams було додано функції профілювання. До виконання запитів до БД фіксувався час за допомогою microtime(true), а після — розраховувався витрачений час та пікове використання пам'яті через memory_get_peak_usage(true)

```php
$start = microtime(true);
$queryCount = 0;

$ttlSeconds = 60;
```

```php
$elapsed = (microtime(true) - $start) * 1000;
$peakMemory = memory_get_peak_usage(true) / 1024 / 1024;

error_log(sprintf('ПІСЛЯ ОПТИМІЗАЦІЇ – Час: %.2f мс, пам\'ять: %.2f МБ, запитів: %d', $elapsed, $peakMemory, $queryCount));
```

Крок 2. Навантажувальне тестування «як є»

За допомогою утиліти Apache Bench (ab) було проведено тестування неоптимізованого коду (імітація 20 паралельних клієнтів, загалом 200 запитів):

.\ab -n 200 -c 20 "http://localhost/practicum07/api.php?resource=teams" Отримані показники базової лінії показали пропускну здатність 579.11 запитів/сек та середній час запиту 34.53 мс.

```
PS F:\Xampp\apache\bin> .\ab -n 200 -c 20 "http://localhost/practicum07/api.php?resource=teams"

Concurrency Level:      20
Time taken for tests:   0.345 seconds
Complete requests:      200
Failed requests:        137
   (Connect: 0, Receive: 0, Length: 137, Exceptions: 0)
Total transferred:      199445 bytes
HTML transferred:       157445 bytes
Requests per second:    579.11 [#/sec] (mean)
Time per request:       34.536 [ms] (mean)
Time per request:       1.727 [ms] (mean, across all concurrent requests)
Transfer rate:          563.97 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.4      0       1
Processing:     3   30  15.7     28      95
Waiting:        3   30  15.7     28      95
Total:          3   31  15.6     28      95

Percentage of the requests served within a certain time (ms)
  50%     28
  66%     33
  75%     38
  80%     40
  90%     48
  95%     63
  98%     83
  99%     94
 100%     95 (longest request)
```

Крок 3. Пошук вузького місця (Проблема N+1)

Для виявлення вузького місця було додано лічильник SQL-запитів $queryCount. Виявлено проблему N+1: спочатку виконувався 1 загальний запит для отримання списку всіх команд, а потім у циклі foreach для кожної команди виконувався окремий запит для підрахунку середніх очок у її місті (SELECT AVG(points) FROM teams WHERE city = :city). Замість 1-2 запитів до БД сервер робив 1 + N запитів.

```
30      foreach ($teams as &$team) {
31          $cityStmt = $pdo->prepare('SELECT AVG(points) FROM teams WHERE city = :city');
32          $cityStmt->execute([':city' => $team['city']]);
33          $team['city_avg_points'] = round((float)$cityStmt->fetchColumn(), 2);
34          $queryCount++;
35      }
```

Крок 4. Додавання індексу на стовпець фільтрації

Для оптимізації пошуку за містами (стовпець city) було перевірено план виконання запиту за допомогою EXPLAIN SELECT * FROM teams WHERE city = 'Київ';. До створення індексу СУБД робила повне сканування таблиці (type = ALL). Після виконання CREATE INDEX idx_teams_city ON teams (city); план запиту показав використання індексу (type = ref), що суттєво прискорило вибірку.

До:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	teams	ALL	*NULL*	*NULL*	*NULL*	*NULL*	4	Using where

Після:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	teams	ref	idx_teams_city	idx_teams_city	1022	const	1	Using index condition

Крок 5. Усунення N+1-запиту

Неоптимізований цикл із SQL-запитами всередині було переписано. Замість окремих запитів було використано один агрегатний запит SELECT city, AVG(points) AS avg_points FROM teams GROUP BY city. Отримані дані зіставлялися з командами на рівні масивів PHP. Кількість SQL-запитів зменшилася з N+1 до рівно 2.

```
$cityAveragesStmt = $pdo->query('SELECT city, AVG(points) AS avg_points FROM teams GROUP BY city');
$cityAverages = $cityAveragesStmt->fetchAll(PDO::FETCH_KEY_PAIR);
$queryCount++;
```

Крок 6. Додавання кешування «важкого» запиту

Для ендпоінта виводу турнірної таблиці було реалізовано файлове кешування. Результат обробки масиву зберігався у тимчасовий файл cache_standings.json із часом життя (TTL) 60 секунд. Також було додано механізм інвалідації: при виконанні POST-запитів (додавання команди або нарахування очок) файл кешу примусово видалявся за допомогою unlink(), щоб наступний GET-запит оновив дані.

```php
if (is_file($cacheFile) && (time() - filemtime($cacheFile)) < $ttlSeconds) {
    $teams = json_decode(file_get_contents($cacheFile), true);
    $fromCache = true;
```

```php
file_put_contents($cacheFile, json_encode($teams));
```

Крок 7. Повторне вимірювання після оптимізацій

Після усунення проблеми N+1 та увімкнення кешування було повторно запущено навантажувальний тест за допомогою Apache Bench. Отримані нові показники: пропускна здатність зросла до 946.14 запитів/сек, а середній час обробки впав до 21.14 мс.

```
PS F:\Xampp\apache\bin> .\ab -n 200 -c 20 "http://localhost/practicum07/api.php?resource=teams"
Server Port:            80

Document Path:          /practicum07/api.php?resource=teams
Document Length:        780 bytes

Concurrency Level:      20
Time taken for tests:   0.211 seconds
Complete requests:      200
Failed requests:        199
   (Connect: 0, Receive: 0, Length: 199, Exceptions: 0)
Total transferred:      197586 bytes
HTML transferred:       155586 bytes
Requests per second:    946.14 [#/sec] (mean)
Time per request:       21.139 [ms] (mean)
Time per request:       1.057 [ms] (mean, across all concurrent requests)
Transfer rate:          912.81 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.3      0       1
Processing:     3   19  22.4      9     128
Waiting:        3   18  22.5      9     128
Total:          3   19  22.5      9     129

Percentage of the requests served within a certain time (ms)
  50%      9
  66%     15
  75%     24
  80%     27
  90%     43
  95%     78
  98%    101
  99%    113
 100%    129 (longest request)
```

Крок 8. Короткий звіт «до / після»

```
/*
ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №7
Варіант 20: Спортивна ліга (teams)

| Показник                     | До оптимізації | Після оптимізації |
| :--------------------------- | :------------- | :---------------- |
| Кількість SQL-запитів        | N+1            | 2 (або 0 з кешу)  |
| Середній час на запит (ab)   | 34.54 мс       | 21.14 мс          |
| Requests per second (ab -c 20)| 579.11        | 946.14            |

Висновок:
Усунення проблеми N+1 шляхом використання GROUP BY city та створення індексу
на стовпець city суттєво зменшило навантаження на базу даних. Додавання
файлового кешування для турнірної таблиці дозволило обробляти дані без
повторних SQL-запитів, що збільшило пропускну здатність застосунку
з 579 до 946 запитів за секунду та відчутно зменшило час відгуку.
*/
```